

Node.js Lecture Notes

1. What is Node.js?

Meaning: Node.js is an open-source, cross-platform JavaScript runtime environment. It allows developers to run JavaScript code outside of a web browser. It is primarily used for building fast, scalable backend applications (servers, APIs).

The Node.js Equation

Node.js is fundamentally: V8 Engine + File Handling + Extra Features (C++ Bindings)

- V8 Engine: This is Google Chrome's ultra-fast JavaScript engine, extracted from the browser. It translates JavaScript into machine code.
- File Handling (I/O): Browsers cannot access your computer's hard drive for security reasons. Node.js adds C++ libraries (like libuv) that allow JavaScript to read, write, and delete files on your system.
- Extra Features: Node provides built-in modules for creating web servers, handling network requests, cryptography, and more.

2. What are Modules?

Meaning: In Node.js, a module is just a single JavaScript file containing related code. Modules allow you to break your application into smaller, manageable, reusable pieces. You export functionality from one file and 'require' (import) it into another.

```
Code:
// In math.js
function add(a, b) { return a + b; }
module.exports = add; // Exporting

// In app.js
const add = require('./math'); // Importing
console.log(add(5, 5));
```

3. The File System (fs) Module: CRUD Operations

Meaning: The built-in 'fs' module allows you to Create, Read, Update, and Delete files on your computer. Node.js provides three ways to do this: Synchronously, via Callbacks (Asynchronous), and via Promises (Modern Asynchronous).

__dirname and path

When working with files, you need to know exactly where you are. `\_\_dirname` is a global variable giving the absolute path to the directory containing the current file. We use the built-in 'path' module to safely join paths together.

Code:

```
const path = require('path');
const filePath = path.join(__dirname, 'data.txt');
console.log('We will operate on:', filePath);
```

A. writeFile (Creating/Updating)

Code:

```
const fs = require('fs');
const fsPromises = require('fs/promises');
const content = 'Hello, this is Node.js!';

// 1. Synchronous (Blocks execution until finished)
fs.writeFileSync('syncFile.txt', content);
console.log('Sync Write Done');

// 2. Callback (Non-blocking)
fs.writeFile('callbackFile.txt', content, (err) => {
  if (err) throw err;
  console.log('Callback Write Done');
});

// 3. Promises (Modern Async/Await)
async function writeWithPromise() {
  try {
    await fsPromises.writeFile('promiseFile.txt',
content);
    console.log('Promise Write Done');
  } catch (err) {
    console.log(err);
  }
}
writeWithPromise();
```

B. readFile (Reading)

Code:

```
// 1. Synchronous ('utf8' converts the binary buffer to a
readable string)
const dataSync = fs.readFileSync('syncFile.txt', 'utf8');
console.log('Sync Read:', dataSync);
```

```

// 2. Callback
fs.readFile('callbackFile.txt', 'utf8', (err, data) => {
  if (err) throw err;
  console.log('Callback Read:', data);
});

// 3. Promises
async function readWithPromise() {
  try {
    const data = await
fsPromises.readFile('promiseFile.txt', 'utf8');
    console.log('Promise Read:', data);
  } catch (err) {
    console.log(err);
  }
}
readWithPromise();

```

C. unlink (Deleting)

Code:

```

// 1. Synchronous
fs.unlinkSync('syncFile.txt');
console.log('Sync Delete Done');

// 2. Callback
fs.unlink('callbackFile.txt', (err) => {
  if (err) throw err;
  console.log('Callback Delete Done');
});

// 3. Promises
async function deleteWithPromise() {
  try {
    await fsPromises.unlink('promiseFile.txt');
    console.log('Promise Delete Done');
  } catch (err) {
    console.log(err);
  }
}
deleteWithPromise();

```

4. Node Package Manager (NPM)

Meaning: NPM is the world's largest software registry. It contains thousands of free, pre-written code packages (libraries) created by other developers that you can download and use in your projects.

- **npm init:** This command initializes a new Node.js project. It asks you a series of questions and generates a 'package.json' file.
- **package.json:** This is the "ID card" or "blueprint" of your project. It stores your project's name, version, and importantly, a list of all the third-party packages your project depends on.
- **npm install <package> (or npm i):** Downloads a specific package from the internet into your project.
- **Dependencies:** The packages listed in your package.json that your app needs to run (e.g., Express, Axios).
- **node_modules:** When you run 'npm install', Node creates this folder. It is where all the actual downloaded package code is stored. (Note: Never upload this folder to GitHub, it is too heavy! Others can recreate it by running 'npm install' using your package.json).

5. The HTTP Module & Creating Web Servers

Node.js has a powerful built-in module called 'http'. This module allows Node.js to transfer data over the Hyper Text Transfer Protocol (HTTP), meaning it can act as a web server.

A web server "listens" for requests coming from a client (like a Google Chrome browser or a frontend app) and sends back a response (like an HTML page or JSON data).

5.1 Creating a Basic Server

Code:

```
const http = require('http'); // 1. Import the built-in HTTP module

// 2. Create the server using http.createServer()
// The callback function runs every single time a request hits the server
const server = http.createServer((req, res) => {

    // req (request): contains info about what the user is asking for (URL, method)
    // res (response): is the object we use to send data back to the user

    // 3. Set the response header (Status 200 means OK,
```

```

content is plain text)
    res.writeHead(200, { 'Content-Type': 'text/plain' });

    // 4. Send the actual content and end the response
    res.end('Hello from the Node.js Server!');
});

// 5. Start the server and tell it to listen on Port 3000
const PORT = 3000;
server.listen(PORT, () => {
    console.log(`Server is running and listening on port
${PORT}...`);
});

```

Explanation:

- `http.createServer()`: This method takes a callback function. Inside this callback, we define exactly how the server should react when someone connects to it.
- `res.writeHead(statusCode, headers)`: Sets the HTTP status code (200 = Success, 404 = Not Found) and tells the browser what kind of data is coming (e.g., text/plain, text/html, application/json).
- `res.end()`: Finalizes the response. The server will hang indefinitely if you forget to call `res.end()`!
- `server.listen()`: Keeps the server running continuously, listening for traffic on a specific port (like 3000).

5.2 Routing in Node.js

Routing means sending different responses based on the URL the user is visiting. We can read the URL requested by the user using `req.url`.

```

Code:
const http = require('http');

const server = http.createServer((req, res) => {
    const url = req.url;

    if (url === '/') {
        res.writeHead(200, { 'Content-Type': 'text/plain' });
        res.end('Welcome to the Home Page!');
    }
    else if (url === '/about') {
        res.writeHead(200, { 'Content-Type': 'text/plain' });
        res.end('This is the About Page.');
```

```

    }
    else {
      // 404 Not Found for any other URL
      res.writeHead(404, { 'Content-Type': 'text/plain' });
      res.end('Error 404: Page Not Found');
    }
  });

  server.listen(3000, () => console.log('Server running...'));

```

5.3 Sending HTML Responses

To send HTML instead of plain text, we simply change the Content-Type header to 'text/html'.

Code:

```

const http = require('http');

const server = http.createServer((req, res) => {
  // Tell the browser to expect HTML
  res.writeHead(200, { 'Content-Type': 'text/html' });

  // Send HTML tags
  res.write('<h1>Welcome to my Node App</h1>');
  res.write('<p>This is rendered as HTML by the
browser.</p>');
  res.end();
});

server.listen(3000);

```

5.4 Sending JSON Responses (Building an API)

When building a backend API, we usually send JSON data instead of HTML. To do this, we set the Content-Type to 'application/json' and use `JSON.stringify()` to convert our JavaScript objects before sending.

Code:

```

const http = require('http');

const server = http.createServer((req, res) => {
  // Sample Database data
  const users = [
    { id: 1, name: 'Alice' },
    { id: 2, name: 'Bob' }
  ];

```

```
    if (req.url === '/api/users') {
      // 1. Set header for JSON
      res.writeHead(200, { 'Content-Type':
'application/json' });

      // 2. Convert JS Array/Object to JSON String and send
it
      res.end(JSON.stringify(users));
    }
  });

  server.listen(3000, () => console.log('API Server running on
port 3000'));
```